SQL

```sql
-- ============================================================
-- AI Idea Forge - Initial Database Schema
-- Version: 1.0
-- ============================================================

-- Enable UUID extension
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- ============================================================
-- Drop Existing Tables (Optional for Development)
-- ============================================================

DROP TABLE IF EXISTS activity_logs CASCADE;
DROP TABLE IF EXISTS agent_results CASCADE;
DROP TABLE IF EXISTS agent_execution CASCADE;
DROP TABLE IF EXISTS analyses CASCADE;

-- ============================================================
-- ANALYSES
-- ============================================================

CREATE TABLE analyses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    title TEXT NOT NULL,
    description TEXT,

    industry TEXT,

    status TEXT NOT NULL DEFAULT 'pending'
        CHECK (status IN ('pending','running','completed','failed')),

    overall_score INTEGER DEFAULT 0
        CHECK (overall_score BETWEEN 0 AND 100),

    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- ============================================================
-- AGENT EXECUTION
-- ============================================================

CREATE TABLE agent_execution (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    analysis_id UUID NOT NULL
        REFERENCES analyses(id)
        ON DELETE CASCADE,
```

```
    agent_name TEXT NOT NULL,

    status TEXT NOT NULL DEFAULT 'pending'
        CHECK (status IN ('pending','running','completed','failed')),

    progress INTEGER NOT NULL DEFAULT 0
        CHECK (progress BETWEEN 0 AND 100),

    confidence INTEGER DEFAULT 0
        CHECK (confidence BETWEEN 0 AND 100),

    message TEXT,

    started_at TIMESTAMPTZ,

    completed_at TIMESTAMPTZ,

    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    UNIQUE (analysis_id, agent_name)
);

-- ============================================================
-- AGENT RESULTS
-- ============================================================

CREATE TABLE agent_results (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    analysis_id UUID NOT NULL
        REFERENCES analyses(id)
        ON DELETE CASCADE,

    agent_name TEXT NOT NULL,

    result_json JSONB NOT NULL DEFAULT '{}'::jsonb,

    tokens_used INTEGER DEFAULT 0,

    execution_time_ms INTEGER DEFAULT 0,

    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- ============================================================
-- ACTIVITY LOGS
-- ============================================================

CREATE TABLE activity_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
    analysis_id UUID NOT NULL
        REFERENCES analyses(id)
        ON DELETE CASCADE,

    agent_name TEXT NOT NULL,

    log_level TEXT NOT NULL DEFAULT 'info'
        CHECK (log_level IN ('info','warning','error')),

    message TEXT NOT NULL,

    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- ============================================================
-- INDEXES
-- ============================================================

CREATE INDEX idx_analysis_status
ON analyses(status);

CREATE INDEX idx_analysis_created
ON analyses(created_at DESC);

CREATE INDEX idx_execution_analysis
ON agent_execution(analysis_id);

CREATE INDEX idx_execution_status
ON agent_execution(status);

CREATE INDEX idx_execution_updated
ON agent_execution(updated_at DESC);

CREATE INDEX idx_results_analysis
ON agent_results(analysis_id);

CREATE INDEX idx_results_agent
ON agent_results(agent_name);

CREATE INDEX idx_logs_analysis
ON activity_logs(analysis_id);

CREATE INDEX idx_logs_created
ON activity_logs(created_at DESC);

CREATE INDEX idx_logs_agent
ON activity_logs(agent_name);

-- ============================================================
-- UPDATED_AT TRIGGER
-- ============================================================
```

```sql
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_analyses_updated_at
BEFORE UPDATE ON analyses
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();

CREATE TRIGGER trg_agent_execution_updated_at
BEFORE UPDATE ON agent_execution
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();

-- ============================================================
-- ROW LEVEL SECURITY
-- ============================================================

ALTER TABLE analyses ENABLE ROW LEVEL SECURITY;
ALTER TABLE agent_execution ENABLE ROW LEVEL SECURITY;
ALTER TABLE agent_results ENABLE ROW LEVEL SECURITY;
ALTER TABLE activity_logs ENABLE ROW LEVEL SECURITY;

-- Development Policies (Replace with authenticated policies in production)

CREATE POLICY "Allow all access - analyses"
ON analyses
FOR ALL
USING (true)
WITH CHECK (true);

CREATE POLICY "Allow all access - agent_execution"
ON agent_execution
FOR ALL
USING (true)
WITH CHECK (true);

CREATE POLICY "Allow all access - agent_results"
ON agent_results
FOR ALL
USING (true)
WITH CHECK (true);

CREATE POLICY "Allow all access - activity_logs"
ON activity_logs
FOR ALL
USING (true)
WITH CHECK (true);
```

```
-- ============================================================
-- END
```